



BRENT OZAR
UNLIMITED®

Fundamentals of Index Tuning

Part 2: Indexing for ORDER BY

Part 2 Slide 1
Slides, demos, chat: BrentOzar.com/training/live

Agenda

How ORDER BY comes into play

Combining WHERE and ORDER BY

TOP exceptions: when ORDER BY goes first

How parameters affect key order



**I'd like to place an
ORDER BY
after two equality searches**

Part 2 Slide 3
Slides, demos, chat: BrentOzar.com/training/live



Bring some order around here

```
SELECT Id, DisplayName, Location
FROM dbo.Users
WHERE DisplayName = 'alex'
      AND Location = 'Seattle, WA'
ORDER BY Reputation;
```



Think back to your 2 earlier indexes.

```
SELECT Id, DisplayName, Location
FROM dbo.Users
WHERE DisplayName = 'alex'
      AND Location = 'Seattle, WA'
ORDER BY Reputation;

CREATE INDEX IX_DisplayName_Location
ON dbo.Users(DisplayName, Location);

CREATE INDEX IX_Location_DisplayName
ON dbo.Users(Location, DisplayName);
```



Add new versions with Reputation

```
CREATE INDEX IX_DisplayName_Location_Reputation
ON dbo.Users(DisplayName, Location, Reputation);

CREATE INDEX IX_Location_DisplayName_Reputation
ON dbo.Users(Location, DisplayName, Reputation);

/* Plus a third idea: */
CREATE INDEX IX_Reputation_DisplayName_Location
ON dbo.Users(Reputation, DisplayName, Location);
```



```
SET STATISTICS IO ON;
GO
SELECT Id, DisplayName, Location
FROM dbo.Users WITH (INDEX = 1) /* Clustered index scan */
WHERE DisplayName = N'alex'
AND Location = N'Seattle, WA'
ORDER BY Reputation;

SELECT Id, DisplayName, Location
FROM dbo.Users WITH (INDEX = IX_DisplayName_Location_Reputation)
WHERE DisplayName = N'alex'
AND Location = N'Seattle, WA'
ORDER BY Reputation;

SELECT Id, DisplayName, Location
FROM dbo.Users WITH (INDEX = IX_Location_DisplayName_Reputation)
WHERE DisplayName = N'alex'
AND Location = N'Seattle, WA'
ORDER BY Reputation;

SELECT Id, DisplayName, Location
FROM dbo.Users WITH (INDEX = IX_Reputation_DisplayName_Location)
WHERE DisplayName = N'alex'
AND Location = N'Seattle, WA'
ORDER BY Reputation;
GO
```

Test 'em

Survey says...

Index	Logical Reads	Total Pages in the Index
Clustered index (white pages)	45,184	45,184
IX_DisplayName_Location_Reputation	4	13,995
IX_Location_DisplayName_Reputation	4	14,486
IX_Reputation_DisplayName_Location	13,996	13,996

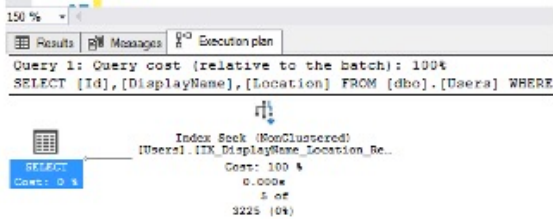
Ouch. Putting reputation first meant no seeking at all, and we scanned the whole thing. (Still better than a table scan though.)



Which one does SQL Server pick?

```
90  /* Which one does SQL Server pick? */
91  SELECT Id, DisplayName, Location
92      FROM dbo.Users
93      WHERE DisplayName = 'alex'
94             AND Location = 'Seattle, WA'
95      ORDER BY Reputation;
96  GO
```

The one that leads with
DisplayName.



ORDER BY

after an INequality search

Part 2 Slide 10
Slides, demos, chat: BrentOzar.com/training/live



Your last query:

```
SELECT Id, DisplayName, Location
FROM dbo.Users
WHERE DisplayName = 'alex'
      AND Location = 'Seattle, WA'
ORDER BY Reputation;
```



Let's go anywhere BUT Seattle

```
SELECT Id, DisplayName, Location
FROM dbo.Users
WHERE DisplayName = 'alex'
      AND Location <> 'Seattle, WA'
ORDER BY Reputation;
```

What's the perfect index for this?
How selective is each part of the filter?



Survey says...

Index	Logical Reads	Total Pages in the Index
Clustered index (white pages)	45,184	45,184
IX_DisplayName_Location_Reputation	13	13,995
IX_Location_DisplayName_Reputation	4,864	14,486
IX_Reputation_DisplayName_Location	13,996	13,996

Ouch. Putting reputation first meant no seeking at all, and we scanned the whole thing. (Still better than a table scan though.)



So the perfect index for it:

```
SELECT Id, DisplayName, Location
FROM dbo.Users
WHERE DisplayName = 'alex'
      AND Location <> 'Seattle, WA'
ORDER BY Reputation;
```

```
CREATE INDEX IX_DisplayName_Location_Reputation ON
dbo.Users (DisplayName, Location, Reputation);
```

Step 1: seek to Alex

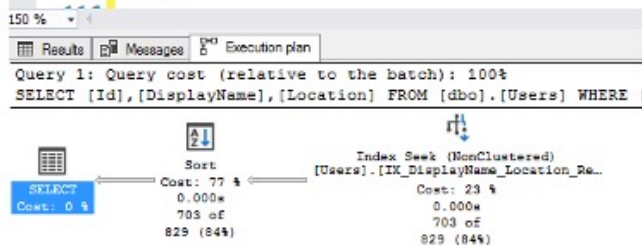
Step 2: scan through,
returning everyone
EXCEPT Seattle

Step 3: read them out
sorted by Reputation,
except...they're not.

Our index gets used, but...

```
110 SELECT Id, DisplayName, Location
111     FROM dbo.Users
112     WHERE DisplayName = 'alex'
113           AND Location <> 'Seattle, WA'
114     ORDER BY Reputation;
115 GO
```

The plan has a Sort even though the data in the index is sorted in order – isn't it?



Write a query to visualize the index

```
CREATE INDEX IX_DisplayName_Location_Reputation ON  
dbo.Users (DisplayName, Location, Reputation);
```

```
SELECT DisplayName, Location, Reputation, Id  
FROM dbo.Users  
ORDER BY DisplayName, Location, Reputation;
```

150 %

Results					Messages					Execution plan				
	DisplayName	Location	Reputation	Id										
1	quico	London, United Kingdom	14747	389099										
2	µBio	California	8999	9796										
3	µlrad	Tehran, Iran	5	136691										
4	0_	NULL	48302	515054										
5	0_o	NULL	1	418884										
6	0_o	NULL	1	438437										
7	0_o	NULL	3	406169										

Part 2 Slide
Slides, dem

Seek down to Alex

```
SELECT DisplayName, Location, Reputation, Id
FROM dbo.Users
ORDER BY DisplayName, Location, Reputation;
```

	DisplayName	Location	Reputation	Id
3..	Arieltibby	Japan	2191	405432
3..	alotao	Paris, France	2099	289493
3..	xixiang	Guangzhou, China	4977	54140
3..	alvando	NULL	16	169640
3..	Alex	NULL	1	72886
3..	Alex	NULL	1	193109
3..	Alex	NULL	1	193346
3..	Alex	NULL	1	202170
3..	Alex	NULL	1	210507
3..	Alex	NULL	1	213524
3..	Alex	NULL	1	213688
3..	Alex	NULL	1	216795
3..	Alex	NULL	1	218333
3..	Alex	NULL	1	220772
3..	Alex	NULL	1	232257
3..	Alex	NULL	1	233850
3..	Alex	NULL	1	236336
3..	Alex	NULL	1	237014
3..	Alex	NULL	1	237016

Remember, we need them ordered by Reputation.

At first it looks like this will work, but...



```
SELECT DisplayName, Location, Reputation, Id
FROM dbo.Users
ORDER BY DisplayName, Location, Reputation;
```

10 %					
Results	Messages	Execution plan			
DisplayName	Location	Reputation	Is		
Alex		701	215550		
Alex		700	200005		
Alex		1133	94019		
Alex		1189	91631		
Alex		1818	79962		
Alex		13733	83185		
Alex	"The Cloud"	405	201706		
Alex	Abenachad, India	161	419711		
Alex	Ableside, NC	3	83742		
Alex	Amsterdam, The Netherl...	121	464950		
Alex	Amway, France	96	383173		
Alex	Athens, Greece	563	423581		
Alex	Atlanta, GA	1490	207090		
Alex	Auckland, New Zealand	6	414011		
Alex	Austin, TX	140	435381		
Alex	Austin, TX	240	177746		
Alex	Australia	817	167011		
Alex	Australia	1473	211869		
Alex	Austria	1935	276637		
Alex	Barcelona, Spain	4967	25787		
Alex	Bay Area, CA, United S...	100	389060		
Alex	Belarus	137	516385		
Alex	Belarus	583	19081		
Alex	Belgium	81	422522		
Alex	Bermuda	1	363997		
Alex	Billinghurst, United K...	1364	181739		
Alex	Boca Raton, FL	6725	257404		
Alex	Bonle, Sweden	71	454570		
Alex	Boston, MA	1130	496645		
Alex	Boston, MA	1965	417291		
Alex	Brazil, Brazil	1825	312808		
Alex	Brisbane, The Netherlands	329	236813		
Alex	Brighton, United Kingd...	53	289608		
Alex	Brooklyn, NY	1231	77204		

Reputation isn't sorted.

We're going to skip everyone who isn't in Seattle.

That means we need all the Alexes on this screen, plus more.

And they're not sorted by Reputation.

The fact that Reputation is "sorted" isn't helping here.



Ordering Reputation doesn't help.

```
SELECT Id, DisplayName, Location
FROM dbo.Users
WHERE DisplayName = 'alex'
      AND Location <> 'Seattle, WA'
ORDER BY Reputation;
```

```
CREATE INDEX IX_DisplayName_Location_Reputation ON
dbo.Users (DisplayName, Location, Reputation);
```

Step 1: seek to Alex

Step 2: scan through,
returning everyone
EXCEPT Seattle

Step 3: read them out
sorted by Reputation,
except...they're not.

To prove it, create another index:

```
CREATE INDEX IX_DisplayName_Location_Reputation  
ON dbo.Users  
(DisplayName, Location, Reputation);  
  
CREATE INDEX IX_DisplayName_Location_Includes ON  
dbo.Users  
(DisplayName, Location) INCLUDE (Reputation);
```



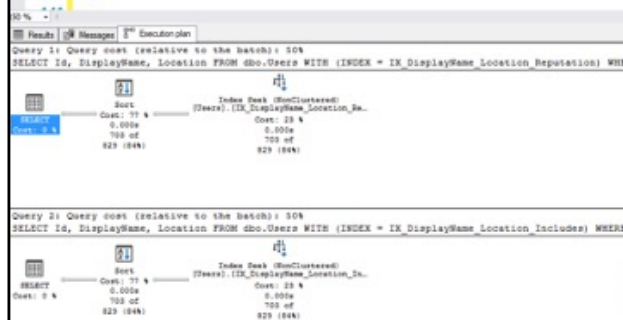
They both get the same plan

```
129 SELECT Id, DisplayName, Location
130 FROM dbo.Users WITH (INDEX = IX_DisplayName_Location_Reputation)
131 WHERE DisplayName = 'alex'
132 AND Location <> 'Seattle, WA'
133 ORDER BY Reputation;
134
135 SELECT Id, DisplayName, Location
136 FROM dbo.Users WITH (INDEX = IX_DisplayName_Location_Includes)
137 WHERE DisplayName = 'alex'
138 AND Location <> 'Seattle, WA'
139 ORDER BY Reputation;
140 GO
```

Use an index hint to test both indexes separately.

Both do the sort.

And both have the same number of logical reads.



Inequality searches make it tricky.

```
WHERE DisplayName = 'alex'  
      AND Location <> 'Seattle, WA'  
ORDER BY Reputation;
```

After you do an inequality search on a field, the sorting of subsequent fields in the index are usually less useful.

(That's a mouthful.)



Putting Reputation **SECOND** helps.

```
SELECT Id, DisplayName, Location
FROM dbo.Users
WHERE DisplayName = 'alex'
      AND Location <> 'Seattle, WA'
ORDER BY Reputation;

CREATE INDEX IX_DisplayName_Reputation_Location ON
dbo.Users (DisplayName, Reputation, Location);
```

Step 1: seek to Alex

Step 2: the sort isn't
needed: they're sorted

Step 3: Skip the users
who aren't in Seattle

The sort is gone with this trick.

```
144 /* Promote Reputation one level: */
145 CREATE INDEX IX_DisplayName_Reputation_Location
146 ON dbo.Users(DisplayName, Reputation, Location);
147 GO
148
149 /* And the sort is gone: */
150 SELECT Id, DisplayName, Location
151 FROM dbo.Users WITH (INDEX = IX_DisplayName_Rep
152 WHERE DisplayName = 'alex'
153 AND Location <> 'Seattle, WA'
154 ORDER BY Reputation;
```

Obscure trick. To get it, key on:

1. Equality fields, then
2. Sort fields, then
3. Inequality fields

SQL Server Enterprise Edition

Results Messages Execution Plan

Query 1: Query cost (relative to the batch): 100%

SELECT Id, DisplayName, Location FROM dbo.Users WITH (INDEX = IX_DisplayName_Reputation_Location) WHERE DisplayName = 'alex' AND Location <> 'Seattle, WA' ORDER BY Reputation;

Index Seek (NonClustered)

(Users): IX_DisplayName_Reputation_Location

Cost: 100 %

0.000s

703 cr

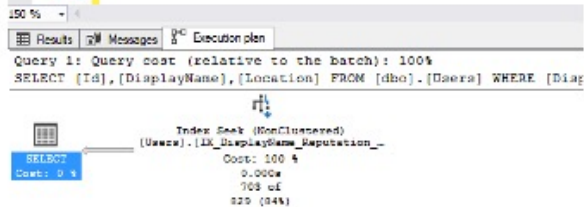
829 (84%)



SQL Server picks it, too.

```
157  /* Which one does SQL Server pick? */
158  SELECT Id, DisplayName, Location
159  FROM dbo.Users
160  WHERE DisplayName = 'alex'
161  AND Location <> 'Seattle, WA'
162  ORDER BY Reputation;
163  GO
164
```

If we don't hint the query, here it picks the index that removes the sort.



What we've learned so far

Indexes help by pre-sorting rows to prep them for:

- WHERE: finding the rows we want
- ORDER BY: sorting them on the way out the door
- GROUP BY, FROM, JOINS, CTEs:
more on these later

And so far, it kinda seems like you want to put keys in that same order: WHERE first, then ORDER BY. But that's not exactly how it works.



S
TOP

me if you've heard this one before

Part 2 Slide 27
Slides, demos, chat: BrentOzar.com/training/live



Start by dropping your indexes.

We're going to tackle a new set of queries, and I don't want to confuse SQL Server's hints with existing indexes.

```
EXEC DropIndexes;
```

Get the code:

BrentOzar.com/go/dropindexes



Design an index for this:

```
SELECT TOP 100 Id, Reputation, CreationDate  
FROM dbo.Users  
WHERE Reputation > 1  
ORDER BY CreationDate ASC;
```



Which field should we lead with?

```
SELECT TOP 100 Id, Reputation, CreationDate
FROM dbo.Users
WHERE Reputation > 1
ORDER BY CreationDate ASC;
```

```
CREATE INDEX IX_Reputation_CreationDate
ON dbo.Users(Reputation, CreationDate);
```

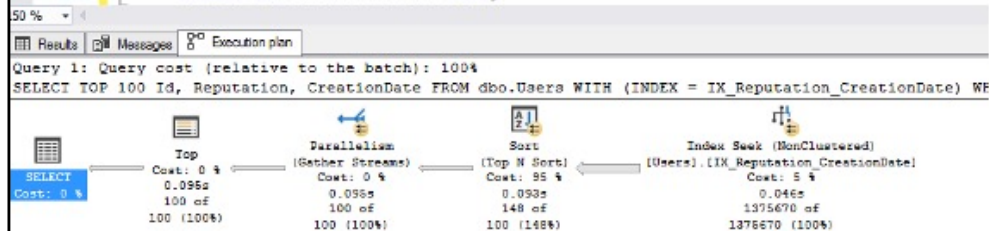
```
CREATE INDEX IX_CreationDate_Reputation
ON dbo.Users(CreationDate, Reputation);
```



If we lead with Reputation...

We seek to 2, but then we find 1.4M users that match!
We have to sort 'em all by CreationDate.

```
187 SELECT TOP 100 Id, Reputation, CreationDate
188 FROM dbo.Users WITH (INDEX = IX_Reputation_CreationDate)
189 WHERE Reputation > 1
190 ORDER BY CreationDate ASC;
```



Visualize the index contents

```
/* Visualizing the Reputation, CreationDate index: */  
SELECT Reputation, CreationDate, Id  
FROM dbo.Users  
ORDER BY Reputation, CreationDate;
```

	Reputation	CreationDate	Id
40711	1	2010-12-31 23:42:32.577	559546
40712	1	2010-12-31 23:46:14.137	559547
40713	1	2010-12-31 23:55:40.483	559553
40714	2	2008-09-15 18:17:21.780	5284
40715	2	2008-11-24 20:50:49.057	40501
40716	2	2008-11-26 09:59:53.243	40566
40717	2	2009-02-03 02:26:11.907	61757
40718	2	2009-02-25 13:51:56.000	70656
40719	2	2009-03-02 09:52:37.567	72731
40720	2	2009-06-24 11:01:29.467	126149
40721	2	2009-09-20 08:21:00.400	164710
40722	2	2009-09-20 10:13:40.473	164767
40723	2	2009-10-13 10:12:12.143	189994
40724	2	2009-10-14 20:04:56.193	190136

When the index is on Reputation, CreationDate, we can seek to 2, but...are the first 10 users we find the lowest CreationDates overall?

Or just the lowest for Reputation = 2?



Easier way to see it

```
/* Visualizing the Reputation, Creati
SELECT Reputation, CreationDate, Id
FROM dbo.Users
ORDER BY Reputation, CreationDate;
```

Results	Messages	Execution plan
Reputation	CreationDate	Id
3819	2010-03-25 08:32:56.610	301514
3819	2010-06-14 02:20:25.290	365977
3819	2010-07-19 15:48:21.980	395875
3820	2009-09-24 08:03:37.333	21537
3820	2009-07-30 09:26:28.910	147695
3820	2010-10-08 09:07:30.453	470062
3821	2009-10-05 08:00:20.540	25234
3821	2008-12-30 00:22:36.087	50025
3821	2010-04-12 15:33:11.823	314670
3821	2010-12-29 08:26:55.083	556899
3822	2009-03-25 01:36:44.977	82333
3822	2010-01-22 20:12:36.790	257065
3822	2010-09-01 10:51:24.297	436853

It's more obvious when we page down to higher Reputation numbers.

The CreationDate keeps resetting with each new Reputation.

The sort on the second field is less useful when we're scanning.



What if we lead with CreationDate?

```
SELECT TOP 100 Id, Reputation, CreationDate
FROM dbo.Users
WHERE Reputation > 1
ORDER BY CreationDate ASC;
```

```
CREATE INDEX IX_Reputation_CreationDate
ON dbo.Users(Reputation, CreationDate);
```



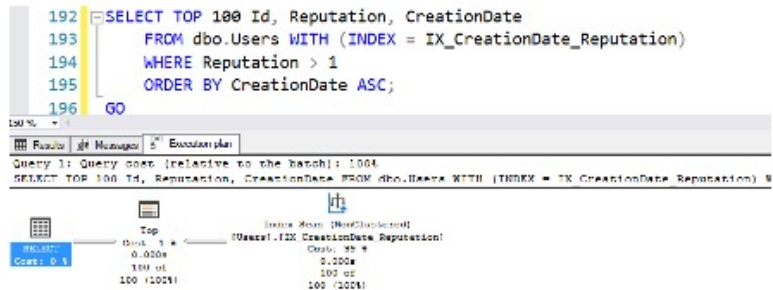
```
CREATE INDEX IX_CreationDate_Reputation
ON dbo.Users(CreationDate, Reputation);
```



We “scan” the index, but...

Remember from How to Think Like the Engine: scan just means we start at one end of the index, and we read until we find the rows that match.

And there’s no sort! The data is already sorted.



Visualize the index contents

```
/* Visualizing the Reputation, CreationDate index: */  
SELECT CreationDate, Reputation, Id  
FROM dbo.Users  
ORDER BY CreationDate, Reputation
```

Results	Msg	Msgpack	W	Execution plan
CreationDate	Reputation	Id		
2008-07-31 00:00:00.000	1	1		
2008-07-31 14:22:31.287	3491	2		
2008-07-31 14:22:31.287	13418	3		
2008-07-31 14:22:31.287	44330	4		
2008-07-31 14:22:31.317	28768	4		
2008-07-31 14:22:31.317	39172	5		
2008-07-31 21:00:24.057	942	8		
2008-07-31 21:08:06.517	14337	9		
2008-07-31 21:57:06.240	101	10		
2008-08-01 00:05:11.147	1890	11		
2008-08-01 04:18:04.843	177138	12		
2008-08-01 12:01:03.661	41167	13		
2008-08-01 12:01:03.023	527	16		
2008-08-01 12:02:21.617	44443	17		
2008-08-01 12:05:14.233	1272	18		
2008-08-01 12:05:11.070	1800	20		
2008-08-01 12:11:11.887	13816	22		
2008-08-01 12:11:43.703	4296	23		
2008-08-01 12:12:53.473	3031	24		
2008-08-01 12:18:23.243	16381	25		
2008-08-01 12:18:14.640	11411	26		
2008-08-01 12:21:40.020	1231	27		
2008-08-01 12:24:53.820	72928	29		

When the index is on
CreationDate, Reputation, we
start reading, looking for 100
users with Reputation > 1.

They almost all match!

As soon as we read 100 rows
that match, we're done. No
need to scan the whole index.



Survey says...

Index	Logical Reads	Total Pages in the Index
Clustered index (white pages)	45,184	45,184
IX_Reputation_CreationDate	3,805	6,812
IX_CreationDate_Reputation	3	6,817



In this case, the ORDER BY field should go first in the index.

```
SELECT TOP 100 Id, Reputation, CreationDate
FROM dbo.Users
WHERE Reputation > 1
ORDER BY CreationDate ASC;
```

```
CREATE INDEX IX_Reputation_CreationDate
ON dbo.Users(Reputation, CreationDate);
```

```
CREATE INDEX IX_CreationDate_Reputation
ON dbo.Users(CreationDate, Reputation);
```





**Remember
selectivity?**

TOP is kinda like a WHERE clause.

```
SELECT TOP 100 Id, Reputation, CreationDate
FROM dbo.Users
WHERE Reputation > 1
ORDER BY CreationDate ASC;
```

That's kinda like saying:

```
SELECT stuff
FROM dbo.Users
WHERE (user is in the top ~100) by CreationDate
```



So let's keep just this one for now

```
DropIndexes;
```

```
CREATE INDEX IX_CreationDate_Reputation  
ON dbo.Users(CreationDate, Reputation);
```

Let's say we decided to just keep this one.



Now run this.

```
SELECT TOP 100 Id, Reputation, CreationDate  
FROM dbo.Users  
WHERE Reputation > 1000000  
ORDER BY CreationDate ASC;
```

There aren't a lot of rows with
Reputation > 1,000,000.



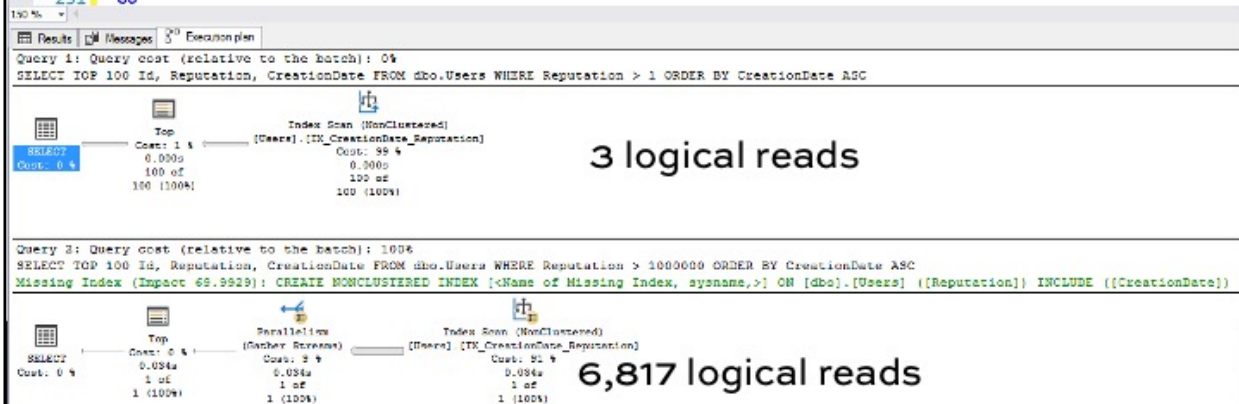
```

220  /* The original query: */
221  SELECT TOP 100 Id, Reputation, CreationDate
222  FROM dbo.Users
223  WHERE Reputation > 1
224  ORDER BY CreationDate ASC;
225
226  /* The new one looking for Jon Skeet: */
227  SELECT TOP 100 Id, Reputation, CreationDate
228  FROM dbo.Users
229  WHERE Reputation > 1000000
230  ORDER BY CreationDate ASC;
231  GO

```

**Both old & new queries
use the index...**

But the index isn't as good of a
fit for the second query. Why?



Jon Skeet isn't in the first 100.

```
SELECT TOP 100 Id, Reputation, CreationDate
FROM dbo.Users
WHERE Reputation > 1000000
ORDER BY CreationDate ASC;
```

The TOP 100 by CreationDate is only selective IF the person you're looking for is in that list.

In this case, WHERE Reputation > 1000000 is much more selective – that should go first.







Index for the WHERE
to reduce reads

Index for the ORDER BY
to reduce sorts

**Indexing requires
compromises and choices.**

These are just 2 inequality searches.

```
SELECT TOP 100 Id, Reputation, CreationDate  
FROM dbo.Users  
WHERE Reputation > 1000000  
ORDER BY CreationDate ASC;
```

It comes down to:

- Which ones are the most selective
- And whether you want to cut reads or cut sorts
- Which parameters run the most often



Say this is a stored procedure.

```
CREATE PROC usp_SearchUsers  
    @SearchReputation INT AS  
  
SELECT TOP 100 Id, Reputation, CreationDate  
FROM dbo.Users  
WHERE Reputation > @SearchReputation  
ORDER BY CreationDate ASC;  
GO
```



```
CREATE PROC usp_SearchUsers  
    @SearchReputation INT AS  
  
SELECT TOP 100 Id, Reputation, CreationDate  
FROM dbo.Users  
WHERE Reputation > @SearchReputation  
ORDER BY CreationDate ASC;  
GO
```

When @SearchReputation = 1, lots of data matches,
so it's better to index on CreationDate, then Reputation.

When @SearchReputation = 1,000,000, then only 1 person matches,
so it's better to index on Reputation, then CreationDate.





Re-cap

Recap

If your WHERE clause is filtering just for equalities, then add the ORDER BY fields into the index key, and the index will handle all the sorting for you.

Out here in the real world, though, your query will have a mix of equality and inequalities.

Different parameter values affect key order too.

Our goal: get a good enough combination of keys to cover as many queries as practical.





BRENT OZAR
UNLIMITED®

Fundamentals of Index Tuning

Lab 2: let's see what you learned about ORDER BY.

Part 2 Slide 52
Slides, demos, chat: BrentOzar.com/training/live

Lab requirements

Download any Stack Overflow database:

- BrentOzar.com/go/querystack
- I'm using the 50GB Stack Overflow 2013 (but any year is fine, even the 10GB one)

Desktop/laptop requirements:

- Any supported SQL Server version will work
- The faster your machine, the faster your indexes will get created



Working through the lab

Read the first query, execute it, do your work inline, taking notes as you go

90 minutes: you work through the lab, asking questions in Slack as you go, and get lunch (either lunch first, or after your work)

The live stream will be off during lunch.

After lunch: I work through it onscreen

